Database

By Dr. Taghinezhad



Scan for More Information:

Website: ataghinezhad.github.io, Email:a0taghinezhad@gmail.com



Chapter 1: Introduction

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Outline

- کاربردهای سیستم پایگاه داده
- هدف از سیستم‌های پایگاه داده
- نمایش داده(View of Data)
- زبان‌های پایگاه داده
- طراحی پایگاه داده
- موتور پایگاه داده
- معماری پایگاه داده
- کاربران و مدیران پایگاه داده
- تاریخچه سیستم‌های پایگاه داده

https://taghinezhad.github.io/



سیستم‌های پایگاه‌داده

- DBMS شامل اطلاعات مربوط به یک کمپانی خاص است:
  - مجموعه‌ای از داده‌های مرتبط به هم.
  - مجموعه‌ای از برنامه‌ها برای دسترسی به داده‌ها.
  - محیطی که هم مناسب و هم کارآمد برای استفاده باشد.

- سیستم‌های پایگاه داده برای مدیریت مجموعه‌هایی از داده‌ها استفاده می‌شوند که:
  - بسیار ارزشمند هستند.
  - نسبتاً بزرگ هستند.
  - توسط چندین کاربر و برنامه، اغلب به طور همزمان، دسترسی می‌شوند.

- یک سیستم پایگاه داده مدرن یک سیستم نرم‌افزاری پیچیده است که وظیفه آن مدیریت یک مجموعه بزرگ و پیچیده از داده‌ها است.

- پایگاه‌های داده تمام جنبه‌های زندگی ما را تحت تأثیر قرار می‌دهند.

https://taghinezhad.github.io/



نمونه‌های کاربرد پایگاه داده

- **اطلاعات سازمانی**
  - فروش: مشتریان، محصولات، خریدها.
  - حسابداری: پرداخت‌ها، دریافت‌ها، دارایی‌ها.
  - منابع انسانی: اطلاعات در مورد کارمندان، حقوق، مالیات بر حقوق.

- **تولید**: مدیریت تولید، موجودی، سفارشات، زنجیره تأمین.

- **بانکداری و امور مالی**
  - اطلاعات مشتری، حساب‌ها، وام‌ها و تراکنش‌های بانکی.
  - تراکنش‌های کارت اعتباری.
  - امور مالی: خرید و فروش ابزارهای مالی (مانند سهام و اوراق قرضه)؛ ذخیره داده‌های بازار در زمان واقعی.

https://taghinezhad.github.io/

- **دانشگاه‌ها:** ثبت‌نام، نمرات



نمونه‌های کاربرد پایگاه داده (ادامه)

- **خطوط هوایی:** رزرو، برنامه‌ها.
- **مخابرات:** سوابق تماس‌ها، پیامک‌ها و مصرف داده، تولید صورت‌حساب‌های ماهانه، حفظ موجودی کارت‌های اعتباری پیش‌پرداخت.
- **خدمات مبتنی بر وب**
  - خرده‌فروشان آنلاین: پیگیری سفارش، توصیه‌های شخصی‌سازی شده.
  - تبلیغات آنلاین.
- **پایگاه‌های داده اسناد (Document databases)**
- **سیستم‌های ناوبری:** برای نگهداری مکان‌های مختلف مورد علاقه همراه با مسیرهای دقیق جاده‌ها، سیستم‌های قطار، اتوبوس‌ها و غیره.

https://taghinezhad.github.io/



هدف از سیستم‌های پایگاه داده

در روزهای اولیه، برنامه‌های پایگاه داده مستقیماً بر روی سیستم‌های فایل ساخته می‌شدند، که منجر به موارد زیر می‌شد:

- **تکرار و عدم سازگاری داده‌ها:** داده‌ها در چندین قالب فایل ذخیره می‌شوند که منجر به تکثیر اطلاعات در فایل‌های مختلف می‌شود.

- **دشواری در دسترسی به داده‌ها**
  - نیاز به نوشتن یک برنامه جدید برای انجام هر کار جدید.

- **انزوای داده(Data isolation)**
  - فایل‌ها و فرمت‌های متعدد.

- **مشکلات یکپارچگی(Integrity problems)**
  - محدودیت‌های یکپارچگی (مانند موجودی حساب > ۰) به جای اینکه صراحتاً بیان شوند، "مدفون" در کد برنامه می‌شوند.
  - افزودن محدودیت‌های جدید یا تغییر محدودیت‌های موجود سخت است.

https://taghinezhad.github.io/



هدف از سیستم‌های پایگاه داده (ادامه)

- **اتمی بودن به‌روزرسانی‌ها(Atomicity of updates)**
  - ممکن است خطاها پایگاه داده را در یک حالت ناسازگار با به‌روزرسانی‌های جزئی انجام شده باقی بگذارند.
  - مثال: انتقال وجه از یک حساب به حساب دیگر باید یا به طور کامل انجام شود یا اصلاً اتفاق نیفتد.
- **دسترسی همزمان توسط چندین کاربر**
  - دسترسی همزمان برای عملکرد مورد نیاز است.
  - دسترسی‌های همزمان کنترل نشده می‌تواند منجر به عدم سازگاری شود.
    - مثال: دو نفر به طور همزمان یک موجودی (مثلاً ۱۰۰) را می‌خوانند و آن را با برداشت پول (مثلاً ۵۰ برای هر کدام) به‌روزرسانی می‌کنند.
- **مشکلات امنیتی**
  - سخت است که به کاربر اجازه دسترسی به برخی از داده‌ها، و نه همه آن‌ها، داده شود.

https://taghinezhad.github.io/

سیستم‌های پایگاه داده راه‌حل‌هایی برای همه مشکلات فوق ارائه می‌دهند.



مثال پایگاه داده دانشگاه

- در این متن ما از یک پایگاه داده دانشگاهی برای توضیح همه مفاهیم استفاده خواهیم کرد.
- داده‌ها شامل اطلاعات در مورد:
  - دانشجویان
  - اساتید
  - کلاس‌ها

- نمونه‌های برنامه کاربردی:
  - افزودن دانشجویان، اساتید و دوره‌های جدید.
  - ثبت‌نام دانشجویان در دوره‌ها و تولید فهرست کلاس‌ها.
  - اختصاص نمرات به دانشجویان، محاسبه میانگین نمرات (GPA) و تولید کارنامه‌ها.

https://taghinezhad.github.io/



نمایش داده (View of Data)

- یک سیستم پایگاه داده مجموعه‌ای از داده‌های مرتبط به هم و مجموعه‌ای از برنامه‌ها است که به کاربران اجازه می‌دهد تا به این داده‌ها دسترسی پیدا کرده و آن‌ها را تغییر دهند.
- یک هدف اصلی از سیستم پایگاه داده ارائه یک **نمای انتزاعی از داده‌ها به کاربران** است.

  - **مدل‌های داده(Data models)**
    - مجموعه‌ای از ابزارهای مفهومی برای توصیف داده‌ها، روابط داده‌ها، معناشناسی داده‌ها و محدودیت‌های سازگاری.

  - **انتزاع داده(Data abstraction)**
    - پنهان کردن پیچیدگی ساختارهای داده برای نمایش داده‌ها در پایگاه داده از کاربران از طریق چندین سطح.

https://taghinezhad.github.io/



مدل‌های داده

- مجموعه‌ای از ابزارها برای توصیف:
  - داده
  - روابط داده
  - معناشناسی داده
  - محدودیت‌های داده

- مدل رابطه‌ای (Relational model)

- مدل داده موجودیت-رابطه(Entity-Relationship data model) :عمدتاً برای طراحی پایگاه داده

- مدل‌های داده مبتنی بر شیء (Object-based data models) :شیء-گرا و شیء-رابطه‌ای

- مدل داده نیمه‌ساخت‌یافته (XML) (Semi-structured data model)

https://taghinezhad.github.io/

- سایر مدل‌های قدیمی‌تر:
  - مدل شبکه‌ای (Network model)
  - مدل سلسله‌مراتبی (Hierarchical model)



مدل رابطه‌ای

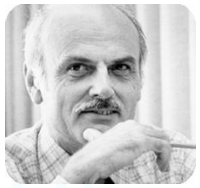
- تمام داده‌ها در جداول مختلف ذخیره می‌شوند.
- مثال داده‌های جدولی در مدل رابطه‌ای

Columns

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows

(a) The *instructor* table



Ted Codd
Turing Award 1981

https://taghinezhad.github.io/



یک نمونه پایگاه داده رابطه‌ای

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

https://taghinezhad.github.io/



سطوح انتزاع

- **سطح فیزیکی(Physical level)** : توصیف می‌کند که یک رکورد (به عنوان مثال، استاد) چگونه ذخیره می‌شود.

- **سطح منطقی(Logical level)** : داده‌های ذخیره‌شده در پایگاه داده و روابط میان داده‌ها را توصیف می‌کند.

```
type instructor = record
        ID : string;
        name : string;
        dept_name : string;
        salary : integer;
end;
```

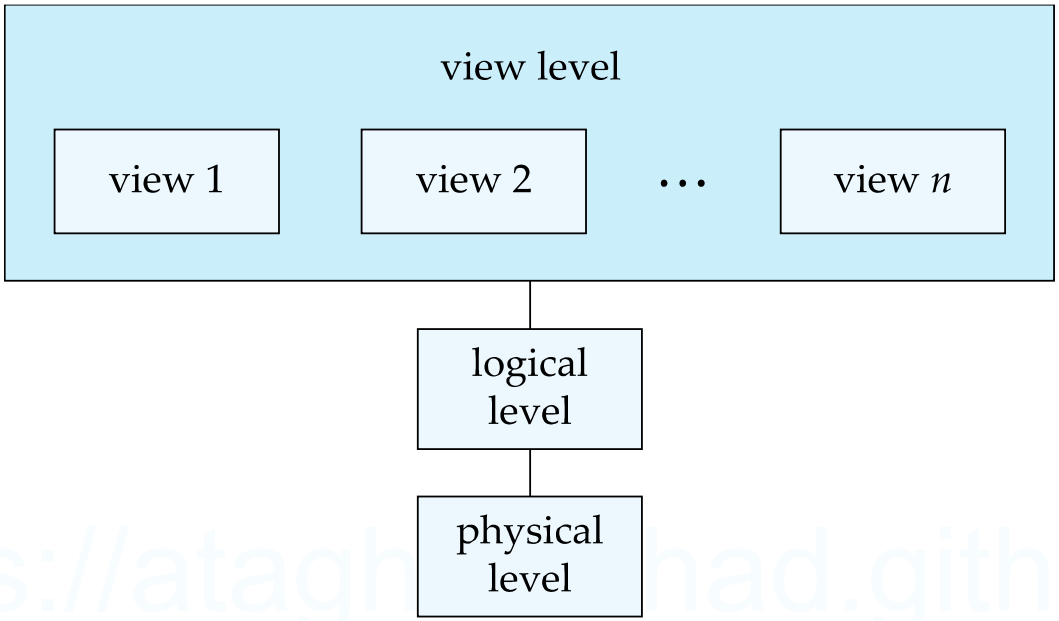
https://taghinezhad.github.io/

- **سطح نما(View level)** : برنامه‌های کاربردی جزئیات انواع داده را پنهان می‌کنند.
  - نماها همچنین می‌توانند اطلاعات (مانند حقوق کارمند) را برای اهداف امنیتی پنهان کنند.



نمای داده (View of Data)

یک معماری برای یک سیستم پایگاه داده



https://taghinezhad.github.io/



نمونه‌ها و طرح‌ها (Instances and Schemas)

- شبیه به انواع (types) و متغیرها (variables) در زبان‌های برنامه‌نویسی.
- **طرح منطقی(Logical Schema)** : ساختار منطقی کلی پایگاه داده.
  - مثال: پایگاه داده شامل اطلاعاتی در مورد مجموعه‌ای از مشتریان و حساب‌ها در یک بانک و رابطه بین آن‌ها است.
    - مشابه اطلاعات نوع (type information) یک متغیر در یک برنامه.
- **طرح فیزیکی(Physical schema)** : ساختار فیزیکی کلی پایگاه داده.
- **نمونه(Instance)** : محتوای واقعی پایگاه داده در یک نقطه زمانی خاص.
  - مشابه با مقدار (value) یک متغیر.

https://taghinezhad.github.io/



استقلال فیزیکی داده (Physical Data Independence)

- **استقلال فیزیکی داده:** توانایی اصلاح طرح فیزیکی بدون تغییر طرح منطقی.

  - برنامه‌های کاربردی به طرح منطقی وابسته هستند.

  - به طور کلی، واسطهای بین سطوح و مولفه‌های مختلف باید به خوبی تعریف شوند تا تغییرات در برخی قسمت‌ها به طور جدی بر دیگر قسمت‌ها تأثیر نگذارد.

https://taghinezhad.github.io/



زبان تعریف داده (Data Definition Language - DDL)

- نماد مشخصات برای تعریف شمای پایگاه داده.
- مثال:

```
create table instructor (
        ID          char(5),
        name        varchar(20),
        dept_name   varchar(20),
        salary      numeric(8,2)
        )
```

- کامپایلر DDL مجموعه‌ای از الگوهای جدول (table templates) را تولید می‌کند که در یک دیکشنری داده (data dictionary) ذخیره می‌شوند.

- دیکشنری داده حاوی فراداده (metadata) است (یعنی داده‌ها در مورد داده‌ها).
  - طرح پایگاه داده.
  - محدودیت‌های یکپارچگی(Integrity constraints) .
    - کلید اصلی(ID) (Primary key) به طور منحصربه‌فرد اساتید را شناسایی می‌کند.
  - احراز هویت(Authorization) .
    - چه کسی می‌تواند به چه چیزی دسترسی داشته باشد.

https://taghinezhad.ir



Data Manipulation Language - زبان دستکاری داده (DML)

- زبانی برای دسترسی و به‌روزرسانی داده‌هایی که توسط مدل داده مناسب سازماندهی شده‌اند.
  - DMLبه عنوان زبان پرس و جو (query language) نیز شناخته می‌شود.

اساساً دو نوع زبان دستکاری داده وجود دارد:

- **DML رویه‌ای(Procedural DML)** : از کاربر می‌خواهد مشخص کند که چه داده‌هایی مورد نیاز است و چگونه آن داده‌ها را بدست آورد.
- **DML اعلانی(Declarative DML)** : از کاربر می‌خواهد مشخص کند که چه داده‌هایی مورد نیاز است بدون تعیین چگونگی بدست آوردن آن داده‌ها.
- DMLهای اعلانی معمولاً یادگیری و استفاده آسان‌تری نسبت به DML های رویه‌ای دارند.
- DMLهای اعلانی به عنوان DML های غیررویه‌ای (non-procedural DMLs) نیز شناخته می‌شوند.
- بخشی از یک DML که شامل بازیابی اطلاعات است، زبان پرس و جو نامیده می‌شود.

https://taghinezhad.github.io/



زبان پرس و جو SQL

- زبان پرس و جو SQL غیررویه‌ای است .یک پرس و جو چندین جدول (احتمالاً فقط یکی) را به عنوان ورودی می‌گیرد و همیشه یک جدول واحد را برمی‌گرداند.

- مثال برای پیدا کردن همه اساتید در دپارتمان علوم کامپیوتر:

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```

- SQL معادل یک ماشین تورینگ نیست.

- برای اینکه بتوان توابع پیچیده را محاسبه کرد، SQLمعمولاً در یک زبان سطح بالاتر تعبیه می‌شود.

- برنامه‌های کاربردی عموماً از طریق یکی از موارد زیر به پایگاه‌های داده دسترسی پیدا می‌کنند:
  - افزونه‌های زبان برای اجازه دادن به SQL تعبیه‌شده(embedded SQL) .
  - واسط برنامه کاربردی (Application program interface) مانند (ODBC/JDBC) که به پرس و جوهای SQL اجازه می‌دهد به پایگاه داده ارسال شوند.

https://taghinezhad.github.io/



دسترسی به پایگاه داده از برنامه کاربردی

- زبان‌های پرس و جو غیررویه‌ای مانند SQL به اندازه یک ماشین تورینگ جهانی قدرتمند نیستند.

- SQL از اعمالی مانند ورودی از کاربران، خروجی به نمایشگرها، یا ارتباط از طریق شبکه پشتیبانی نمی‌کند.

- چنین محاسبات و اعمالی باید در یک زبان میزبان(host language) ، مانند C/C++ ، Java یا Python ، با پرس و جوهای SQL تعبیه‌شده که به داده‌ها در پایگاه داده دسترسی پیدا می‌کنند، نوشته شوند.

https://taghinezhad.github.io/

- برنامه‌های کاربردی (Application programs) برنامه‌هایی هستند که برای تعامل با پایگاه داده به این شیوه استفاده می‌شوند.



طراحی پایگاه داده

فرآیند طراحی ساختار کلی پایگاه داده:

- **طراحی منطقی(Logical Design)** : تصمیم‌گیری در مورد طرح‌واره پایگاه داده .طراحی پایگاه داده مستلزم یافتن مجموعه‌ای "خوب" از طرح‌های رابطه (relation schemas) است.

  - تصمیم کسب و کار: چه صفاتی (attributes) را باید در پایگاه داده ثبت کنیم؟
  - تصمیم علوم کامپیوتر: چه طرح‌واره‌های رابطه‌ای باید داشته باشیم و چگونه صفات باید بین طرح‌واره‌های مختلف رابطه توزیع شوند؟

https://taghinezhad.github.io/

- **طراحی فیزیکی(Physical Design)** : تصمیم‌گیری در مورد چیدمان فیزیکی پایگاه داده.



موتور پایگاه داده (Database Engine)

- یک سیستم پایگاه داده به ماژول‌هایی تقسیم می‌شود که هر یک از مسئولیت‌های سیستم کلی را مدیریت می‌کنند.

- مولفه‌های عملیاتی (**functional components**) یک سیستم پایگاه داده را می‌توان به موارد زیر تقسیم کرد:

  - **مدیر ذخیره‌سازی** (The storage manager)

  - **مولفه پردازشگر پرس و جو** (The query processor component)

  - **مولفه مدیریت تراکنش** (The transaction management component)

https://taghinezhad.github.io/



مدیر ذخیره‌سازی (Storage Manager)

- یک ماژول برنامه‌ای است که واسطی بین داده‌های سطح پایین ذخیره شده در پایگاه داده و برنامه‌های کاربردی و پرس و جوهای ارسال شده به سیستم فراهم می‌کند.

- مدیر ذخیره‌سازی مسئول وظایف زیر است:

  - تعامل با مدیر فایل سیستم عامل(OS file manager) .
  - ذخیره‌سازی، بازیابی و به‌روزرسانی کارآمد داده‌ها.

- مولفه‌های مدیر ذخیره‌سازی شامل:

  - مدیر احراز هویت و یکپارچگی (Authorization and integrity manager)
  - مدیر تراکنش (Transaction manager)
  - مدیر فایل (File manager)
  - مدیر بافر(Buffer manager)

https://taghinezhad.github.io/



مدیر ذخیره‌سازی (ادامه)

- مدیر ذخیره‌سازی چندین ساختار داده را به عنوان بخشی از پیاده‌سازی سیستم فیزیکی پیاده‌سازی می‌کند:
  - **فایل‌های داده(Data files)** : خود پایگاه داده را ذخیره می‌کنند.
  - **دیکشنری داده(Data dictionary)** : فراداده در مورد ساختار پایگاه داده، به ویژه طرح‌واره پایگاه داده، را ذخیره می‌کند.
  - **شاخص‌ها(Indices)** : می‌توانند دسترسی سریع به اقلام داده را فراهم کنند.
    - یک شاخص پایگاه داده به آن اقلام داده‌ای اشاره می‌کند که یک مقدار خاص را نگه می‌دارند.

https://taghinezhad.github.io/



پردازشگر پرس و جو (Query Processor)

- مولفه‌های پردازشگر پرس و جو شامل:

  - **مفسر DDL (DDL interpreter)** : دستورات DDL را تفسیر کرده و تعاریف را در دیکشنری داده ثبت می‌کند.

  - **کامپایلر DML (DML compiler)** : دستورات DML را در یک زبان پرس و جو به یک طرح ارزیابی (evaluation plan) ترجمه می‌کند که شامل دستورالعمل‌های سطح پایینی است که موتور ارزیابی پرس و جو (query evaluation engine) می‌فهمد.

    - کامپایلر DML بهینه‌سازی پرس و جو (query optimization) را انجام می‌دهد؛ یعنی کم‌هزینه‌ترین طرح ارزیابی را از بین گزینه‌های مختلف انتخاب می‌کند.

  - **موتور ارزیابی پرس و جو (Query evaluation engine)** :
    دستورالعمل‌های سطح پایین تولید شده توسط کامپایلر DML را اجرا می‌کند.

https://taghinezhad.github.io/

پردازش پرس‌وجو (Query Processing) در DBMS

DBMS دستور SQL را دریافت می‌کند، بهترین روش اجرای آن را انتخاب می‌کند و در نهایت نتیجه را برمی‌گرداند.



مثال: پیدا کردن نام دانشجویان رشته CS

```
SELECT Name
FROM Students
WHERE Department = 'CS';
```

جدول Students

StudentID	Name	Age	Department
1	Ali	20	CS
2	Sara	22	IT
3	Reza	19	CS
4	Mina	21	Math

1 Parsing and Translation (تجزیه و ترجمه)

- بررسی صحت دستور SQL
- اعتبار جدول Students و ستون‌های Department = Name
- بررسی مجوز دسترسی کاربر

2 Query Optimization (بهینه‌سازی پرس‌وجو)

انتخاب روش اجرا:

- استفاده از ایندکس روی Department (در صورت وجود)
- یا Table Scan (بررسی کل جدول)

3 Query Execution (اجرای پرس‌وجو)

- خواندن داده‌ها از جدول یا ایندکس
- فیلتر کردن رکوردهای با 'CS' = Department
- استخراج ستون Name

4 Result (نتیجه)

- بازگرداندن نام دانشجویان:
  - Ali
  - Reza

چرا بهینه‌سازی مهم است؟

برای مثال، دو روش احتمالی برای اجرای پرس‌وجو وجود دارد:

Table Scan (بررسی کامل جدول)

- خواندن تمام رکوردهای جدول
- مناسب وقتی جدول کوچک است.
- یا بیشتر رکوردها مطابق شرط هستند.

Index Scan (استفاده از ایندکس)

- استفاده از ایندکس روی Department
- کاهش تعداد خواندن‌ها
- اما همیشه ارزان‌تر نیست.

هدف اصلی Query Optimization کاهش هزینه اجرای پرس‌وجو است.



مدیریت تراکنش (Transaction Management)

- یک **تراکنش** (transaction) مجموعه‌ای از عملیات است که یک تابع منطقی واحد را در یک برنامه پایگاه داده انجام می‌دهد.

- **مولفه مدیریت تراکنش** اطمینان حاصل می‌کند که پایگاه داده علی‌رغم خرابی‌های سیستم (مانند قطع برق و خرابی‌های سیستم عامل) و خرابی‌های تراکنش، در یک حالت سازگار (صحیح) باقی می‌ماند.

- **مدیر کنترل همروندی** (Concurrency-control manager) تعامل بین تراکنش‌های همزمان را کنترل می‌کند تا از سازگاری پایگاه داده اطمینان حاصل شود.

https://taghinezhad.github.io/



معماری پایگاه داده

- **پایگاه‌های داده متمرکز(Centralized databases)**
  - یک تا چند هسته، حافظه مشترک.
- **مشتری-سرویس‌دهنده(Client-server)**
  - یک دستگاه سرویس‌دهنده کار را به نمایندگی از چندین دستگاه مشتری اجرا می‌کند.
- **پایگاه‌های داده موازی(Parallel databases)**
  - حافظه مشترک چند هسته‌ای.
  - دیسک مشترک(Shared disk) .
  - بدون اشتراک(Shared nothing) .
- **پایگاه‌های داده توزیع شده(Distributed databases)**
  - توزیع جغرافیایی.
  - عدم یکنواختی طرح‌واره/داده(Schema/data heterogeneity)

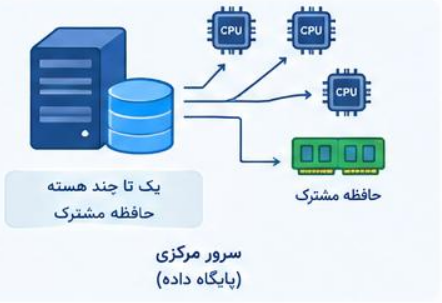
https://taghinezhad.github.io/

انواع معماری پایگاه داده

معماری پایگاه داده مشخص می‌کند داده‌ها کجا ذخیره می‌شوند، پردازش‌ها چگونه انجام می‌شوند و سیستم‌ها چگونه با یکدیگر ارتباط برقرار می‌کنند.

1 (Centralized databases) (Centralized databases)

در این معماری، اطلاعات در یک سیستم مرکزی ذخیره و مدیریت می‌شوند و پردازش پرس‌وجوها نیز در همان سیستم انجام می‌شود. این سیستم می‌تواند یک یا چند هسته پردازنده داشته باشد و هسته‌ها از حافظه اصلی مشترک استفاده کنند.



مثال:
یک دانشگاه تمام اطلاعات دانشجویان، استادان و نمرات را روی یک سرور مرکزی نگهداری می‌کند.

محدودیت: خرابی سیستم مرکزی می‌تواند دسترسی به کل پایگاه داده را مختل کند.

مزیت: سادگی: مدیریت و پشتیبان‌گیری ساده است.

2 سرویس‌دهنده (Client-server)

در این معماری، چند دستگاه مشتری درخواست‌های خود را برای یک دستگاه سرویس‌دهنده ارسال می‌کنند. سرویس‌دهنده درخواست‌ها را پردازش می‌کند، در صورت نیاز با پایگاه داده ارتباط می‌گیرد و نتیجه را به مشتری برمی‌گرداند.



مثال:
در یک سامانه بانکی، مشتری از طریق اپلیکیشن موبایل درخواست مشاهده موجودی حساب خود را می‌دهد. سرور درخواست را دریافت می‌کند، موجودی را از پایگاه داده می‌خواند و نتیجه را به موبایل ارسال می‌کند.

محدودیت: اگر سرور از کار بیفتد یا ظرفیت کافی نداشته باشد، سرویس‌دهی با مشکل مواجه می‌شود.

مزیت: جندین کاربر می‌توانند از یک سرویس مشترک استفاده کنند.

3 پایگاه‌های داده موازی (Parallel databases)

در پایگاه داده موازی، چند پردازنده یا گره پردازشی به‌طور هم‌زمان روی بخش‌های مختلف یک پرس‌وجو یا مجموعه داده کار می‌کنند تا زمان اجرای عملیات کاهش یابد.

الف) حافظه مشترک (Shared Memory)



مثال: یک سرور با چند هسته CPU که همه به RAM مشترک دسترسی دارند.

مزیت: ارتباط ساده بین پردازنده‌ها

محدودیت: گلوگاه حافظه و پهنای باند.

ب) دیسک مشترک (Shared Disk)

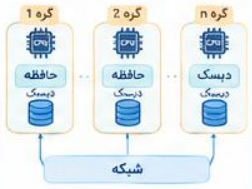


مثال: دو سرور پردازشی که از یک ذخیره‌سازی مشترک استفاده می‌کنند.

مزیت: دسترسی همه گره‌ها به داده‌ها

محدودیت: هماهنگی دسترسی و رقابت بر سر دیسک

ج) بدون اشتراک (Shared Nothing)



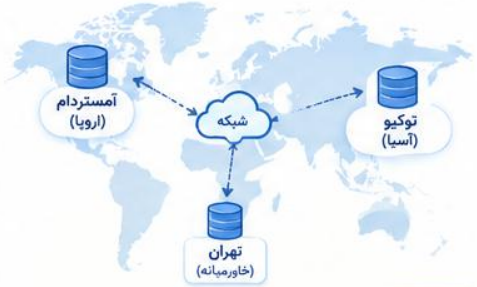
مثال: یک پایگاه داده تحلیلی که اطلاعات مشتریان را بین چند سرور تقسیم می‌کند.

مزیت: افزایش ظرفیت با افزودن گره‌های جدید.

محدودیت: نیاز به تقسیم مناسب داده‌ها و شبکه

4 پایگاه‌های داده توزیع شده (Distributed databases)

در این معماری، داده‌ها در چند سیستم یا مکان مختلف قرار دارند و سیستم‌ها از طریق شبکه با یکدیگر ارتباط برقرار می‌کنند. این مکان‌ها ممکن است در یک ساختمان، چند شهر یا چند کشور باشند.



یک شرکت بین‌المللی شعبه‌هایی در آمستردام، تهران و توکیو دارد. هر شعبه اطلاعات محلی خود را نگهداری می‌کند، اما سامانه مرکزی باید بتواند اطلاعات فروش تمام شعب را برای تهیه گزارش جهانی ترکیب کند.

توزیع جغرافیایی
داده‌ها در مکان‌های مختلف نگهداری می‌شوند تا دسترسی محلی سریع‌تر شود یا سرویس در مناطق مختلف در دسترس باشد.

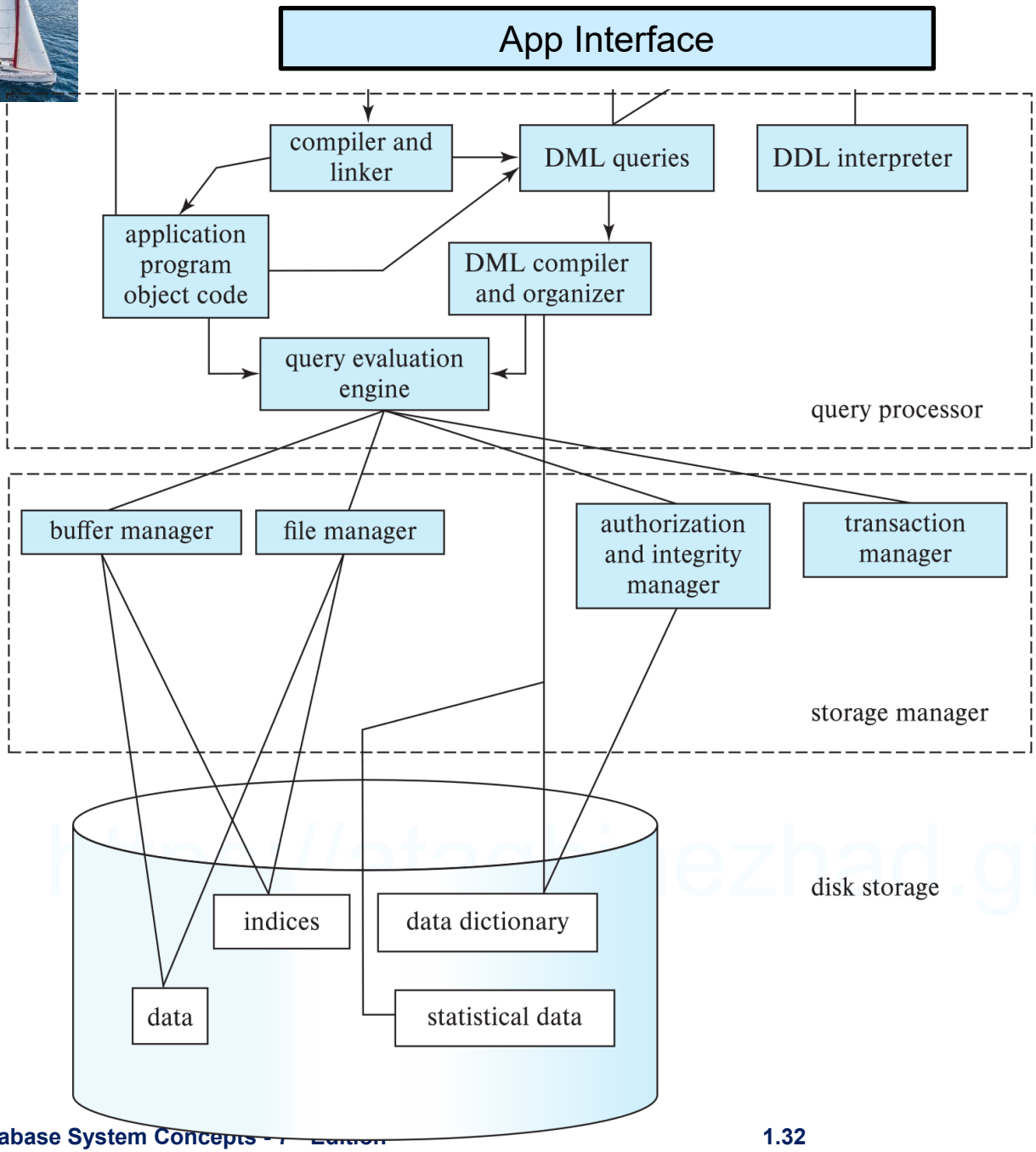
عدم یکنواختی طرح‌واره‌داده (Schema/data heterogeneity)
سیستم‌های مختلف ممکن است ساختار جدول‌ها، قالب داده‌ها یا حتی نوع DBMS متفاوتی داشته باشند.

مزیت: افزایش دسترس‌پذیری، نزدیک بودن داده‌ها به کاربران محلی

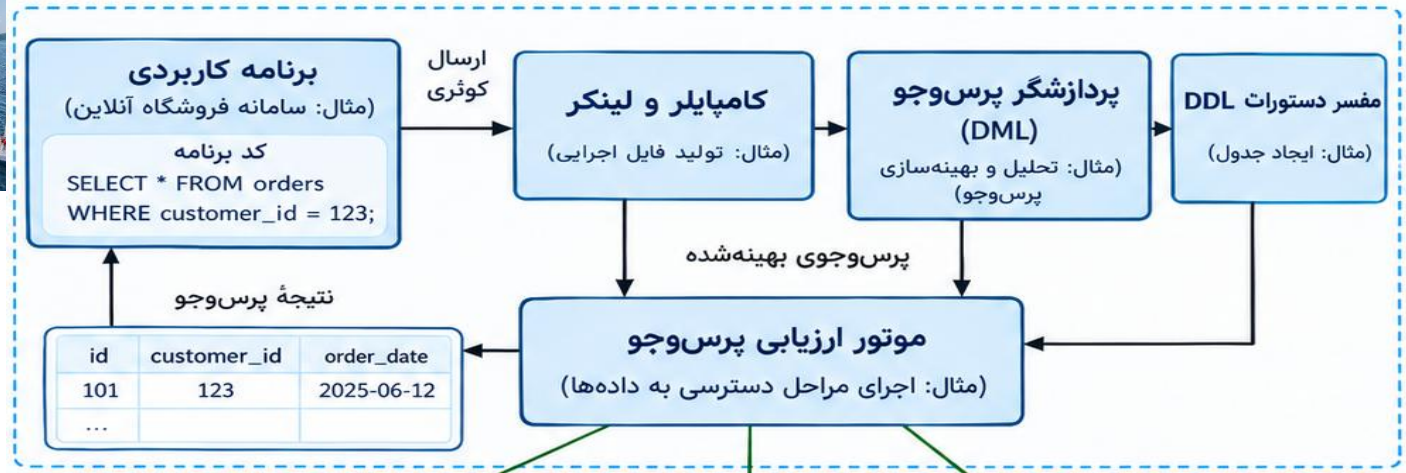
محدودیت: هماهنگی داده‌ها، تاخیر شبکه و مدیریت تفاوت ساختارها



معماری پایگاه داده (متمرکز/حافظه مشترک)



http://taghinezhad.github.io/



پرس‌وجو (Query Processor) دُیستوری

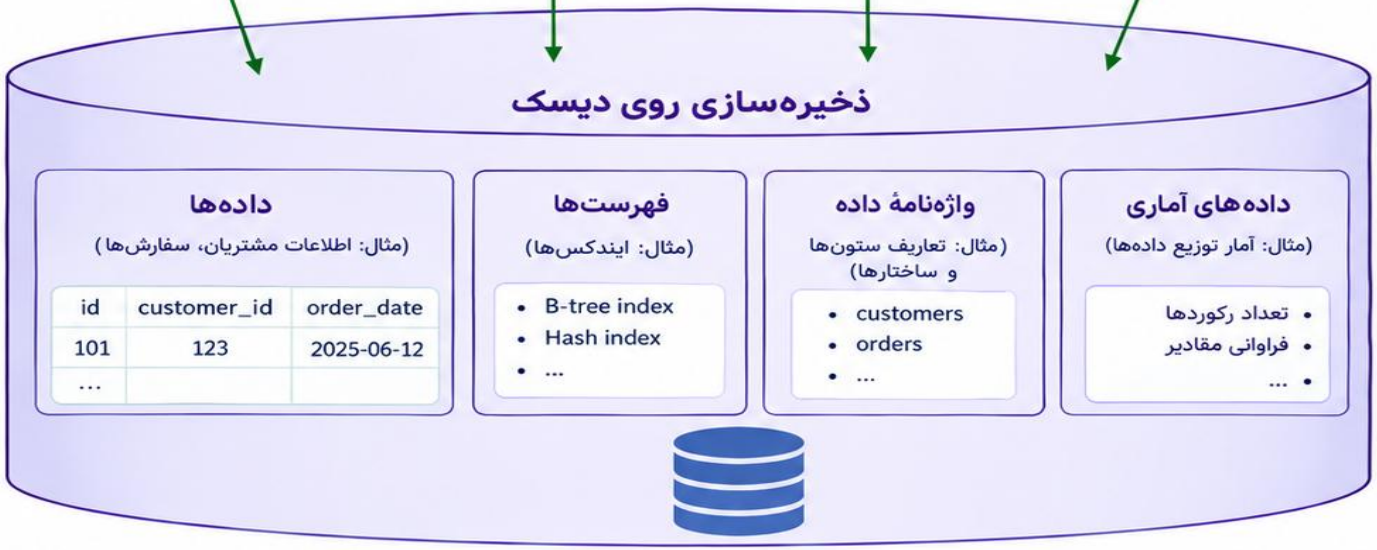
این بخش وظیفهٔ تحلیل، بهینه‌سازی و اجرای پرس‌وجوها را بر عهده دارد.

مثال: پرس‌وجوی بالا به یک برنامهٔ اجرایی بهینه تبدیل می‌شود و موتور ارزیابی‌آن را اجرا می‌کند.



مدیر ذخیره‌ول (Storage Manager)

این بخش واسط بین پردازشگر پرس‌وجو و ذخیره‌سازی داده‌هاست و وظایف مدیریتی مانند بافر، فایل، مجوزها و تراکنش‌ها را انجام می‌دهد.



ذخیره‌سازی‌دیسک (Disk Storage)

در این لایه داده‌ها و ساختارهای کمکی در دیسک ذخیره می‌شوند.

مثال:

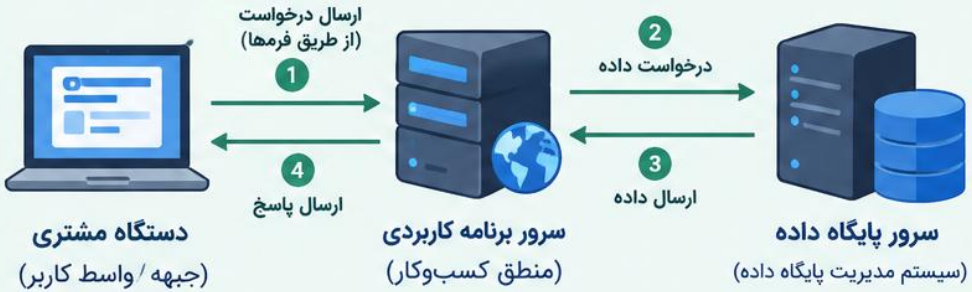
- جدول customers
- ایندکس روی customer_id
- واژه‌نامه داده (Data Dictionary)
- آمارهای جمع‌آوری‌شده

معماری‌های رایج برنامه‌های کاربردی پایگاه داده

معماری سه لایه

در معماری سه لایه، دستگاه مشتری به عنوان یک جبهه (front end) عمل می‌کند و هیچ فراخوانی مستقیم به پایگاه داده ندارد.

- مشتری با یک سرور برنامه کاربردی (application server)، معمولاً از طریق یک واسط فرمها، ارتباط برقرار می‌کند.
- سرور برنامه کاربردی به نوبه خود برای دسترسی به داده‌ها با یک سیستم پایگاه داده ارتباط برقرار می‌کند.

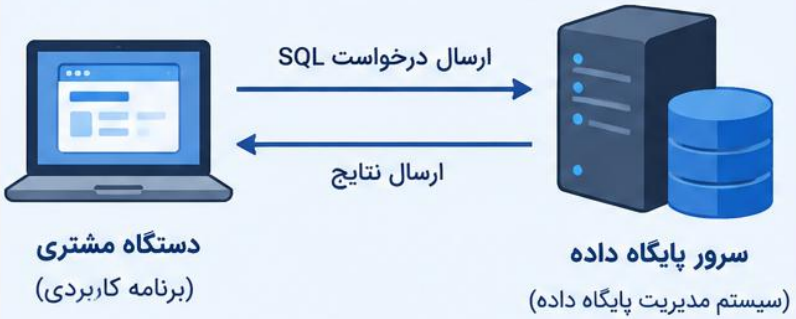


جریان کار:

1. مشتری درخواست خود را از طریق واسط فرمها به سرور برنامه کاربردی ارسال می‌کند.
2. سرور برنامه کاربردی برای دسترسی به داده‌ها، درخواست را به پایگاه داده ارسال می‌کند.
3. پایگاه داده داده‌های مورد نظر را بازمی‌گرداند.
4. سرور برنامه کاربردی پاسخ را به مشتری ارسال می‌کند.

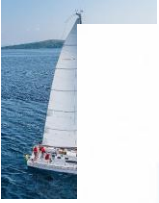
معماری دو لایه

در معماری دو لایه، برنامه در دستگاه مشتری قرار دارد جایی که عملکرد سیستم پایگاه داده را در دستگاه سرویس‌دهنده فراخوانی می‌کند.



ویژگی اصلی:

برنامه مستقیماً با سیستم پایگاه داده در سرور ارتباط برقرار می‌کند.



کاربران پایگاه داده و اجزای پردازشگر پرس‌وجو

کاربران عادی (کارمند آموزش)
استفاده از
فرم ثبت نمره

برنامه‌نویسان کاربردی
استفاده از
برنامه سامانه آموزشی

کاربران حرفه‌ای (تحلیل‌گران)
نوشتن
ابزار اجرای پرس‌وجو (SQL)

مدیر پایگاه داده
استفاده از
ابزار مدیریت پایگاه داده

پردازشگر پرس‌وجو

کامپایلر و پیوندهنده
کد اجرایی برنامه
پرس‌وجوهای DML (مانند SELECT و UPDATE)
کامپایلر و بهینه‌ساز DML
موتور ارزیابی پرس‌وجو

مفسر DDL (مانند CREATE TABLE)

مثال: پایگاه داده دانشگاه

کارمند آموزش: ثبت نمره دانشجو
برنامه آموزشی: تبدیل درخواست به دستور پایگاه داده
پردازش و اجرای دستور
به‌روزرسانی جدول نمرات

نمونه دستور SQL:

```
SELECT  نام، نمره
FROM    نمرات
WHERE   شماره_دانشجو - ۱۲۳;
```





مدیر پایگاه داده (Database Administrator - DBA)

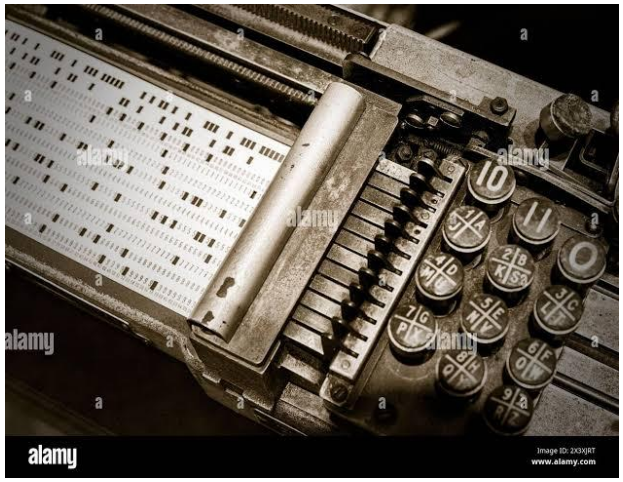
شخصی که کنترل مرکزی بر سیستم دارد، مدیر پایگاه داده (DBA)نامیده می‌شود.
وظایف یک DBA شامل:

- **تعریف طرح Schema.**
- **تعریف ساختار ذخیره‌سازی** و روش دسترسی به پایگاه داده و جداول.
- **اصلاح طرح و سازمان فیزیکی.**
- **اعطای مجوز دسترسی به داده‌ها.**
- **نگهداری روتین:**
  - **پشتیبان‌گیری** دوره‌ای از پایگاه داده.
  - **اطمینان از در دسترس بودن فضای کافی دیسک آزاد** برای عملیات عادی و ارتقاء فضای دیسک در صورت نیاز.
  - **نظارت بر کارهایی** که در پایگاه داده در حال اجرا هستند.

https://taghinezhad.github.io/



تاریخچه سیستم‌های پایگاه داده



- **دهه ۱۹۵۰ و اوایل دهه ۱۹۶۰:**
  - پردازش داده با استفاده از نوارهای مغناطیسی برای ذخیره‌سازی.
    - نوارها فقط دسترسی ترتیبی (sequential access) را فراهم می‌کردند.
  - کارت‌های سوراخ‌دار (Punched cards) برای ورودی.
- **اواخر دهه ۱۹۶۰ و دهه ۱۹۷۰:**
  - دیسک‌های سخت اجازه دسترسی مستقیم به داده‌ها را دادند.
  - مدل‌های داده شبکه‌ای و سلسله‌مراتبی به طور گسترده‌ای مورد استفاده قرار گرفتند.
  - تد کاد (Ted Codd) مدل داده رابطه‌ای را تعریف کرد.
    - برای این کار برنده جایزه تورینگ ACM شد.
    - IBM Research نمونه اولیه System R را آغاز کرد.
    - UC Berkeley (Michael Stonebraker) نمونه اولیه Ingres را آغاز کرد.
    - Oracle اولین پایگاه داده رابطه‌ای تجاری را منتشر کرد.
  - پردازش تراکنش با عملکرد بالا (برای آن دوران).

https://taghinezhad.github.io/



تاریخچه سیستم‌های پایگاه داده (ادامه)

- **دهه ۱۹۸۰:**
  - نمونه‌های اولیه رابطه‌ای تحقیقاتی به سیستم‌های تجاری تکامل یافتند.
    - SQLبه استاندارد صنعتی تبدیل شد.
  - سیستم‌های پایگاه داده موازی و توزیع‌شده.
    - Teradata. ،IBM ،Wisconsin
  - سیستم‌های پایگاه داده شیء-گرا.
- **دهه ۱۹۹۰:**
  - برنامه‌های کاربردی بزرگ پشتیبانی از تصمیم‌گیری (decision support) و داده‌کاوی(data-mining) .
  - انبارهای داده (data warehouses) بزرگ چند ترابایتی.
  - ظهور تجارت وب.

https://taghinezhad.github.io/



تاریخچه سیستم‌های پایگاه داده (ادامه)

- **دهه ۲۰۰۰:**
  - سیستم‌های ذخیره‌سازی داده‌های بزرگ(Big data storage systems) .
    - Google BigTable، Yahoo PNuts، Amazon .
    - سیستم‌های"NoSQL" .
  - تجزیه و تحلیل داده‌های بزرگ: فراتر ازSQL .
    - Map reduceو موارد مشابه.
- **دهه ۲۰۱۰:**
  - SQLبارگذاری مجدد شد(SQL reloaded) .
    - جبهه (SQL front end) SQL به سیستم‌هایMap Reduce .
    - سیستم‌های پایگاه داده موازی گسترده (Massively parallel database systems) .
    - پایگاه‌های داده درون حافظه (main-memory databases) چند هسته‌ای.

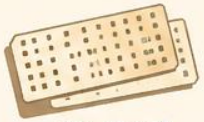
https://dataginezhad.github.io/

تاریخچه و تکامل سیستم‌های پایگاه داده

دهۀ ۱۹۵۰ و اوایل دهۀ ۱۹۶۰



نوارهای مغناطیسی
(دسترسی ترتیبی)



کارت‌های پانچ
(ورود داده و دستورها)

- ذخیره‌سازی و پردازش داده‌ها عمدتاً با نوارهای مغناطیسی انجام می‌شد.
- نوارهای مغناطیسی فقط امکان دسترسی ترتیبی به داده‌ها را فراهم می‌کردند.
- از کارت‌های پانچ برای ورود داده‌ها و دستورها استفاده شد.

مثال:
پردازش فهرست حقوق کارکنان با استفاده از نوار مغناطیسی و کارت‌های پانچ

اواخر دهۀ ۱۹۶۰ و دهۀ ۱۹۷۰



دیسک‌های سخت
(دسترسی مستقیم)



مدل‌های داده
(سلسله‌مراتبی و شبکه‌ای)



Ted Codd
مدل رابطه‌ای

IBM System R | UC Berkeley Ingres | ORACLE

- استفاده از دیسک‌های سخت و دسترسی مستقیم به داده‌ها
- مدل‌های دادهٔ سلسله‌مراتبی و شبکه‌ای
- معرفی مدل رابطه‌ای توسط تد کاد
- توسعهٔ نمونهٔ اولیهٔ System R توسط IBM
- شروع توسعهٔ Ingres در دانشگاه برکلی توسط مایکل استون‌بریکر
- عرضهٔ نخستین نسخهٔ تجاری پایگاه دادهٔ رابطه‌ای توسط Oracle
- افزایش قابلیت‌های تراکنش‌ها و کاربردهای تجاری

مثال:
سیستم بانکی با تراکنش‌های آنلاین و همزمان

دهۀ ۱۹۸۰



پایگاه داده‌های موازی
و توزیع‌شده



پایگاه داده‌های شیءگرا

- تبدیل نمونه‌های پژوهشی به سیستم‌های تجاری
- استاندارد شدن SQL
- گسترش پایگاه داده‌های موازی و توزیع‌شده
- پژوهش‌های دانشگاه ویسکانسین و همکاری IBM و Teradata
- توسعهٔ پایگاه داده‌های شیءگرا

مثال:
مدیریت اطلاعات مشتریان با SQL در یک پایگاه دادهٔ تجاری

دهۀ ۱۹۹۰



پشتیبانی از تصمیم‌گیری
(Decision Support)



داده‌کاوی
(Data Mining)



انبار داده
(Data Warehouse)



تجارت الکترونیکی
(E-Commerce)

- گسترش استفاده از پایگاه داده‌ها در برنامه‌های تصمیم‌گیری بزرگ
- داده‌کاوی برای استخراج الگوها و اطلاعات ارزشمند
- توسعهٔ انبارهای داده با ظرفیت چند ترابایتی
- افزایش نیاز به سیستم‌های مقیاس‌پذیر با گسترش اینترنت و تجارت الکترونیکی

مثال:
تحلیل رفتار مشتریان برای بازاریابی هدفمند

دهۀ ۲۰۰۰



Google Bigtable | Yahoo! PNUTS | amazon Dynamo



NoSQL



MapReduce

- رشد سریع حجم داده‌ها و توسعهٔ سیستم‌های کلان‌داده
- معرفی سیستم‌هایی مانند Google Bigtable , Yahoo! PNUTS و Amazon Dynamo
- گسترش خانوادهٔ NoSQL
- نیاز به تحلیل‌های فراتر از SQL سنتی
- توسعهٔ چارچوب MapReduce برای پردازش توزیع‌شدهٔ داده‌ها

مثال:
پردازش داده‌های کاربران وب در مقیاس جهانی

دهۀ ۲۰۱۰



نسل جدید سیستم‌های تحلیلی
(دوباره SQL)

MPP



پردازش موازی گسترده
(Massively Parallel Processing)



پایگاه داده‌های
درون‌حافظه‌ای
(In-Memory)

- بازگشت SQL در قالب نسل جدید سیستم‌های تحلیلی و پردازشی
- ابزارهای SQL Front برای اجرای پرس‌وجو روی سیستم‌های توزیع‌شده و MapReduce
- گسترش پایگاه داده‌های MPP
- افزایش استفاده از پایگاه داده‌های درون‌حافظه‌ای با بهره‌گیری از پردازنده‌های چندهسته‌ای

مثال:
تحلیل داده‌های بزرگ با SQL روی یک پلتفرم MPP با حافظهٔ اصلی

پردازش ترتیبی (نوار مغناطیسی) ← دسترسی مستقیم (دیسک سخت) ← مدل‌های رابطه‌ای و سیستم‌های تجاری ← تصمیم‌گیری و داده‌کاوی ← کلان‌داده و NoSQL ← تحلیل پیشرفته و پردازش موازی

از ذخیره‌سازی سادهٔ داده‌ها تا سیستم‌های مقیاس‌پذیر، هوشمند و تحلیلی



تحولات پایگاه داده‌ها در دهه ۲۰۲۰ تا کنون

فناوری‌های جدید، معماری‌های نوین و هوش مصنوعی، دنیای پایگاه داده‌ها را متحول کرده‌اند.

1 پایگاه داده‌های بومی ابر

پایگاه داده‌های بومی ابر برای مقیاس‌پذیری، دسترس‌پذیری بالا و مدیریت انعطاف‌پذیر منابع توسعه بیشتری یافتند.



Amazon Aurora , Google Cloud Spanner , Azure Cosmos DB

2 تفکیک ذخیره‌سازی از پردازش (Compute–Storage Disaggregaation)

این معماری‌ها به سیستم‌ها اجازه دادند منابع پردازشی و ذخیره‌سازی را مستقل‌تر مقیاس‌دهی کنند.



Kubernetes + Persistent Volumes , Dell PowerEdge

3 معماری‌های Lakehouse

تلاش کردند انعطاف‌پذیری دریاچه‌های داده را با قابلیت‌های مدیریتی و تحلیلی انبارهای داده ترکیب کنند.



مثال: Delta Lake , Databricks Unity Catalog

4 پایگاه داده‌های برداری

برای بازیابی معنایی و کاربردهای هوش مصنوعی، پایگاه‌داده‌های برداری و قابلیت جست‌وجوی برداری در پایگاه داده‌های موجود توسعه یافتند.



Pinecone , Weaviate , pgvector

5 هوش مصنوعی مولد و LLMها

با گسترش هوش مصنوعی مولد و مدل‌های زبانی بزرگ، جست‌وجوی برداری، تولید تقویت‌شده با بازیابی (RAG) و اتصال مدل‌های هوش مصنوعی به داده‌های سازمانی اهمیت بیشتری پیدا کردند.



ChatGPT + Enterprise Data , Azure OpenAI

6 پایگاه داده‌های چندمدلی

امکان مدیریت چند نوع داده، مانند داده‌های رابطه‌ای، اسناد، گراف‌ها و داده‌های مکانی را در یک سامانه یا بستر یکپارچه فراهم کردند.



مثال: MongoDB , Azure Cosmos DB

7 قابلیت‌های هوش مصنوعی

قابلیت‌های هوش مصنوعی برای بهینه‌سازی پرس‌وجوها، کمک به مدیریت پایگاه داده و ساخت برنامه‌های داده‌محور به محصولات مختلف اضافه شدند.



Oracle AI , Amazon RDS with AI , SQL Server Copilot

8 SQL و ابزارهای تخصصی

SQL همچنان زبان مهمی برای مدیریت و تحلیل داده‌ها باقی ماند و در کنار آن، ابزارهای تخصصی برای جست‌وجوی برداری، داده‌های گرافی و پردازش توزیع‌شده گسترش یافتند.



PostgreSQL (pgvector) , Neo4j , Apache Spark





End of Chapter 1

https://ataghinezhad.github.io/